

■ 2.4.3 Working with Operators

You can think of an expression like $f[x]$ as being formed by applying an *operator* f to the expression x . You can get the sum of two expressions x and y just by writing $x + y$. If you have two operators f and g , it is sometimes also useful to think about finding their sum.

Here is a sum of two expressions, that you can think of the results of applying the “operators” f and g to x .

```
In[1]:= f[x] + g[x]
Out[1]= f[x] + g[x]
```

This expression represents the application of the sum of the operators f and g to x .

```
In[2]:= (f + g)[x]
Out[2]= (f + g)[x]
```

You can use `Through` to convert the expression to this form.

```
In[3]:= Through[%, Plus]
Out[3]= f[x] + g[x]
```

`Identity` is a very simple function that just returns its argument. It is useful when you try to build up operators.

```
In[4]:= Identity[x]
Out[4]= x
```

This corresponds to the mathematical operator $1 + \frac{\partial}{\partial x}$.

```
In[5]:= Identity + (D[#, x] &)
Out[5]= Identity + (D[#1, x] & )
```

Mathematica does not automatically apply the separate pieces of the operator to an expression.

```
In[6]:= % [x^2]
Out[6]= (Identity + (D[#1, x] & )) [x^2]
```

You can use `Through` to apply the operator.

```
In[7]:= Through[%, Plus]
Out[7]= 2 x + x^2
```

`Through` does nothing if the operation that appears in the head of the expression does not match the one you specify.

```
In[8]:= Through[(c f)[x], Plus]
Out[8]= (c f)[x]
```

<code>Identity[expr]</code>	the identity function
<code>Through[p[f₁, f₂][x], q]</code>	give $p[f_1[x], f_2[x]]$ if p is the same as q
<code>Operate[p, f[x]]</code>	give $p[f][x]$

Operations for working with operators.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

This has a complicated expression as a head.

```
In[9]:= t = ((1 + a)(1 + b))[x]
Out[9]= ((1 + a) (1 + b))[x]
```

Functions like `Expand` do not automatically go inside heads of expressions.

```
In[10]:= Expand[%]
Out[10]= ((1 + a) (1 + b))[x]
```

`MapAll` also does not go inside heads of expressions.

```
In[11]:= MapAll[Expand, t]
Out[11]= ((1 + a) (1 + b))[x]
```

Rules applied with `/.` do work inside the heads of expressions.

```
In[12]:= t /. a->1
Out[12]= (2 (1 + b))[x]
```

You can use `Operate` to apply functions to the heads of expressions.

```
In[13]:= Operate[p, t]
Out[13]= p[(1 + a) (1 + b)][x]
```

This performs the expansion inside the head of `t`.

```
In[14]:= Operate[Expand, t]
Out[14]= (1 + a + b + a b)[x]
```

When you work with operators, it is important to remember that *Mathematica* always evaluates the head of an expression before it evaluates the rest of the expression. You can thus for example give rules to change the head of an expression, so that the operations that are done on its arguments are different.

This sets `hh[1]` to be the symbol `Plus`.

```
In[15]:= hh[1] = Plus
Out[15]= Plus
```

If you use `hh[1]` as the head of an expression, the elements get added together.

```
In[16]:= hh[1][b, a, c]
Out[16]= a + b + c
```